

# Road Network Planning and Navigation

---

## Road Network Planning and Navigation

1. Course Content
2. Running the Road Network Planning and Navigation Function
  - 2.1 Start Docker Container
  - 2.2 Start Sensors and Odometry
  - 2.3 Start the Road Network Planning and Navigation Node
  - 2.4 Navigation Between Waypoints
  - 2.5 Navigation When the Starting Position is Not on the Road Network
3. Source Code Analysis
  - 3.1 Analysis of route\_bridge.py
  - 3.2 Analysis of conver\_speed.py
4. Navigation Parameter Debugging
  - 4.1 Navigation Controller Debugging
  - 4.2 Obstacle Avoidance and Costmap Debugging

## 1. Course Content

---

Basic: Use a marked road network file for inter-road network navigation on a map.

Advanced: Master the debugging methods for navigation parameter files.

## 2. Running the Road Network Planning and Navigation Function

---

### 2.1 Start Docker Container

- Start the terminal for the humble version of Docker.

```
start_muto_humble
```

```

❑ Serial device detected: /dev/myserial
❑ Speech device detected: /dev/myspeech
❑ Audio device detected: /dev/snd
❑ LiDAR device detected: /dev/rplidar
❑ Input device detected: /dev/input
❑ Video device detected: /dev/video0
❑ Video device detected: /dev/video1
>>> Select image...

>>> Launch container...

Launch Info
Image:          192.168.2.51:5000/muto-humble:latest
Network Mode:  host
Service Ports: 80, 9090, 8888, 8080
X11 Env:        DISPLAY=localhost:10.0

Container started successfully! Container ID: dfec5843dca

>>> Configure camera devices...
>>> Configure audio...
✓ Found Audio Device: (Card 3)
>>> Ready for use...

Access Methods
Enter container: docker exec -it dfec5843dca08bf749c276018f6757ff11d072219e64d796013e6c33e977
fad /bin/bash

Container ready!

-----
ROS_DOMAIN_ID: 38
my_robot_type: Muto | my_lidar: a1
-----
root@raspberrypi:~#

```

## 2.2 Start Sensors and Odometry

- First, let's start the humble version of Docker.

```
start_muto_docker
```

- After Docker starts, launch the laser fusion node and odometry fusion.

```
ros2 launch yahboomcar_nav laser_odom_bringup_launch.py
```

## 2.3 Start the Road Network Planning and Navigation Node

- If the ros2\_kilted docker container is not running, start it first.

```
~/kilted_docker_start.sh
```

- Start the road network planning and navigation node.

```

ros2 launch road_net_route roadnet_nav2.launch.py
map:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map.yaml
1
pose_map:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map
roadnet_file:=/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/road_map.geojson

```

[!TIP]

Optional startup parameter description

**map** Name of the grid map to be loaded.

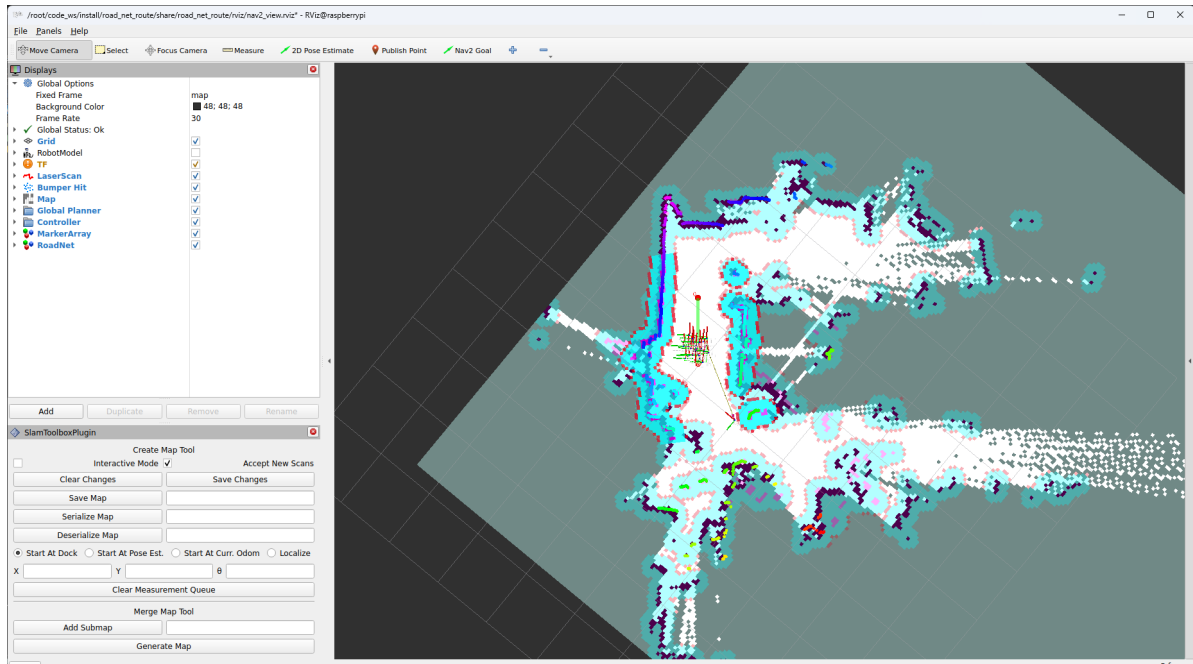
**pose\_map** Name of the pose map file to be loaded. When the global localization method for navigation is slam\_toolbox pure localization, the pose map must be provided.

**params\_file** Navigation parameter file, the default value is

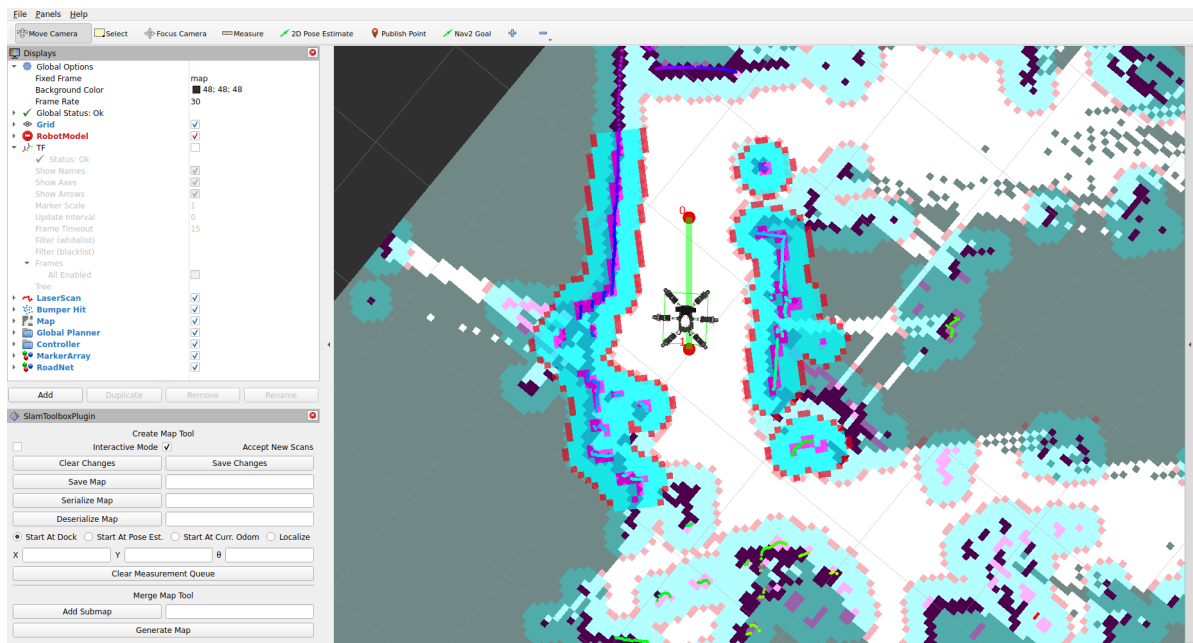
```
os.path.join(get_package_share_directory('road_net_route'), 'config',  
'nav2_kilted.yaml')
```

**roadnet\_file** Road network description file. Note that the full path + filename must be provided for the roadnet\_file.

**localization\_mode** Global localization method, defaults to slam\_toolbox.



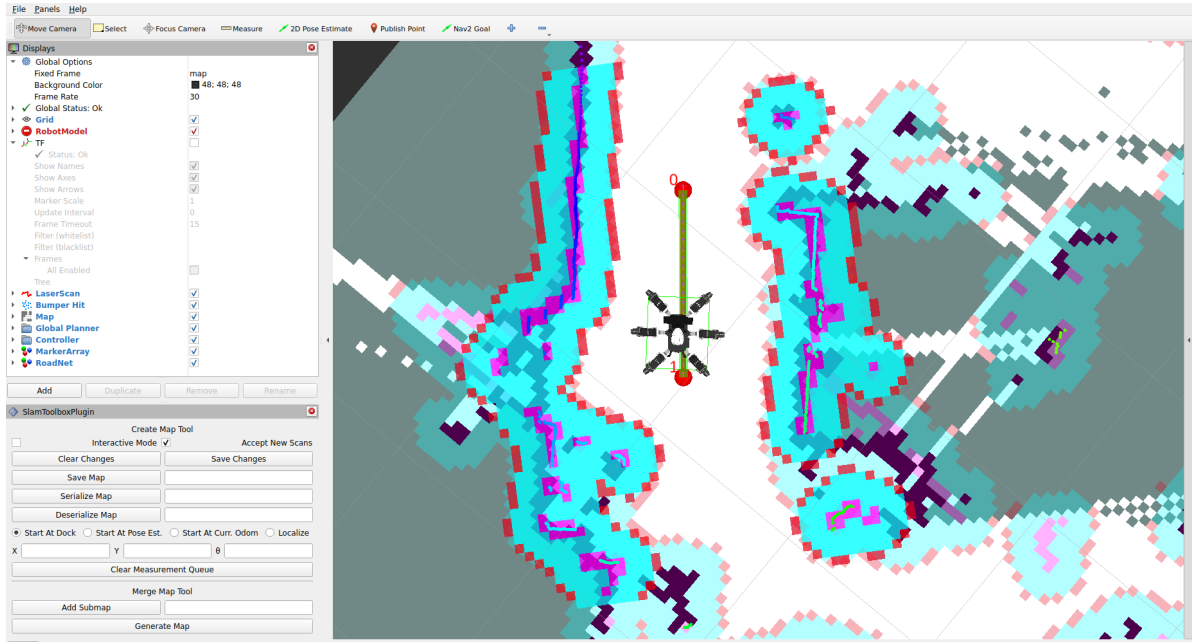
- To speed up the rviz launch, the robot model is not loaded by default. If you need to display the robot model, check the RobotModel in the left tab.



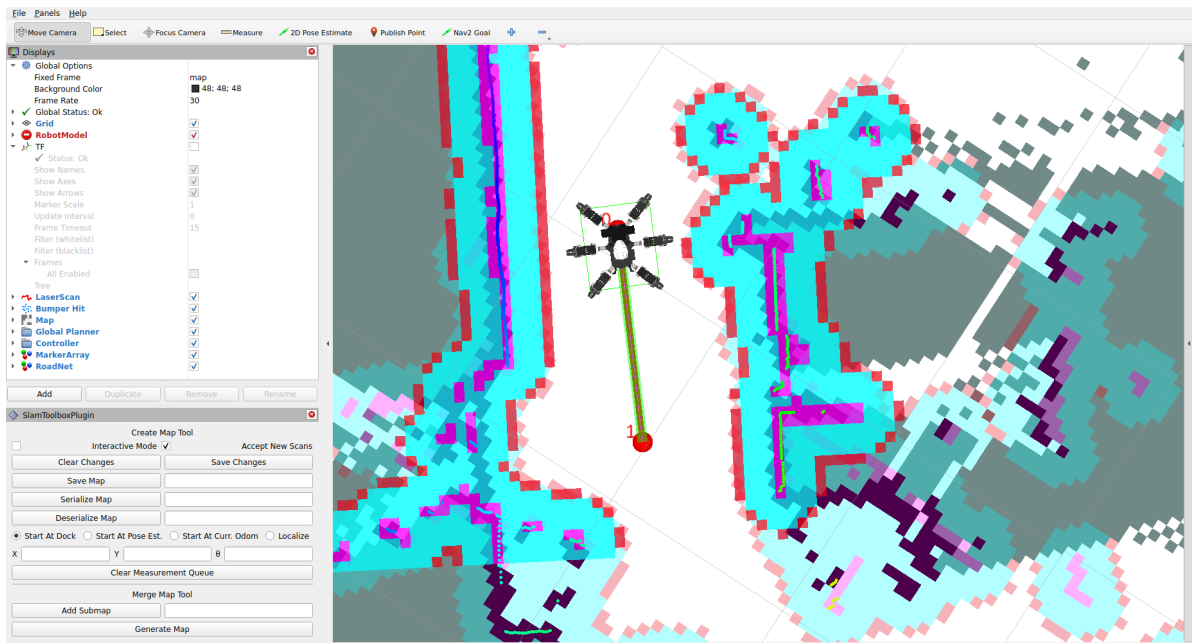
## 2.4 Navigation Between Waypoints

- Here we use road network navigation by publishing the `/road_net_nav_id` topic in the terminal. For example, in the figure, our robot car is close to waypoint 1, and we need to navigate to waypoint 0.

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [1, 0]}"
```



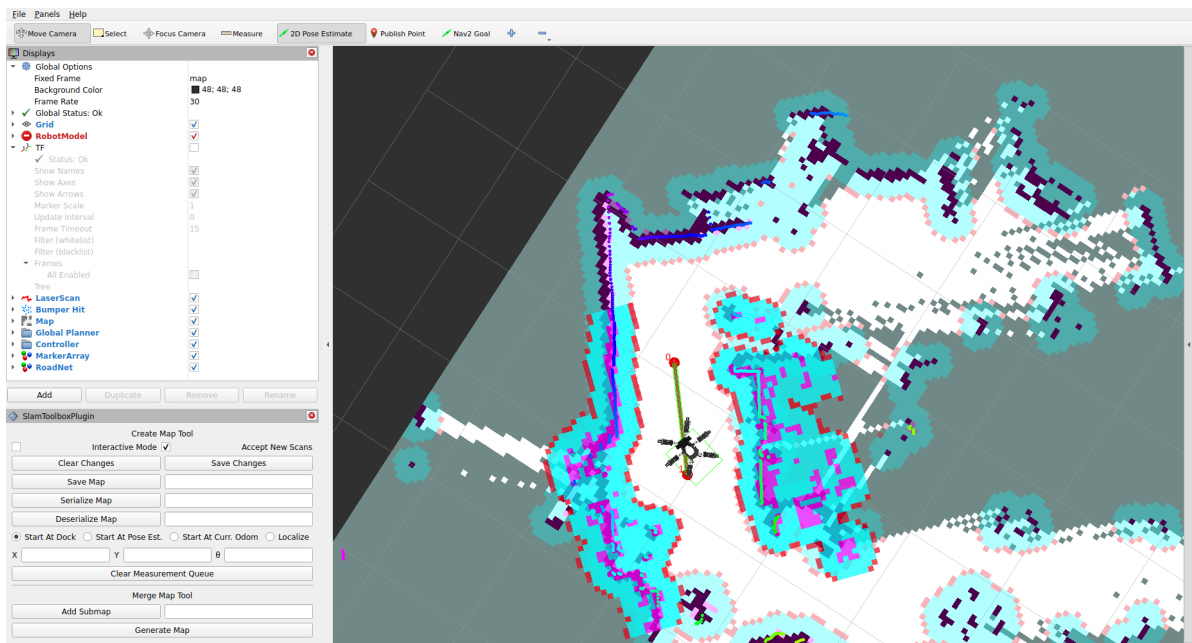
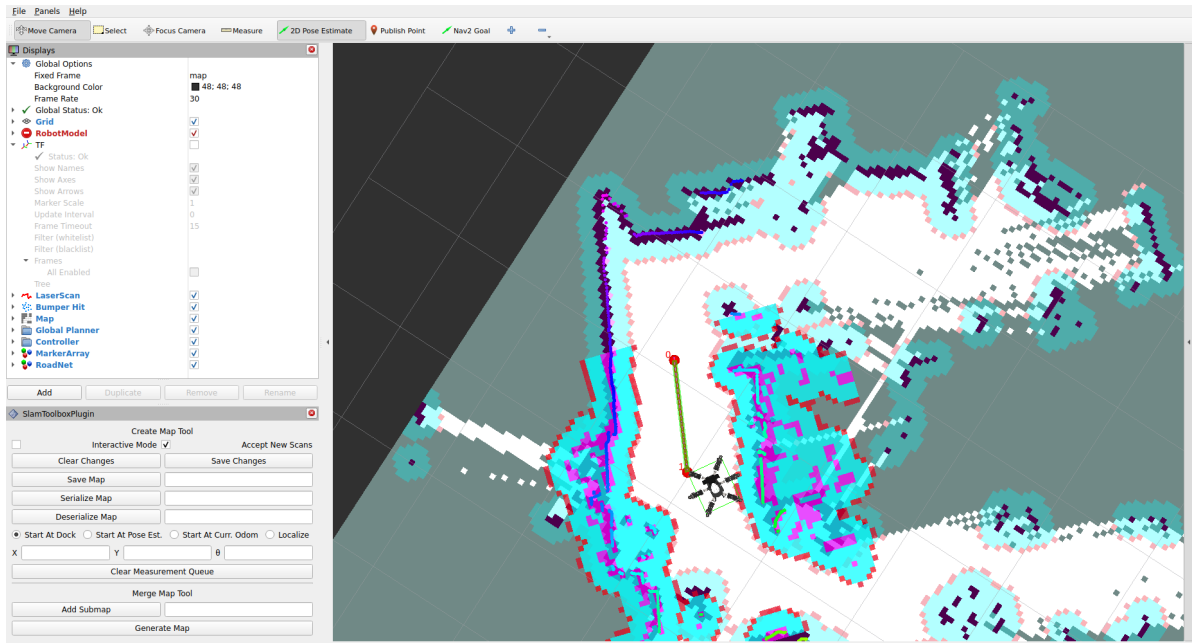
- Then you can see in rviz that the planned global path will only be planned within the road network.

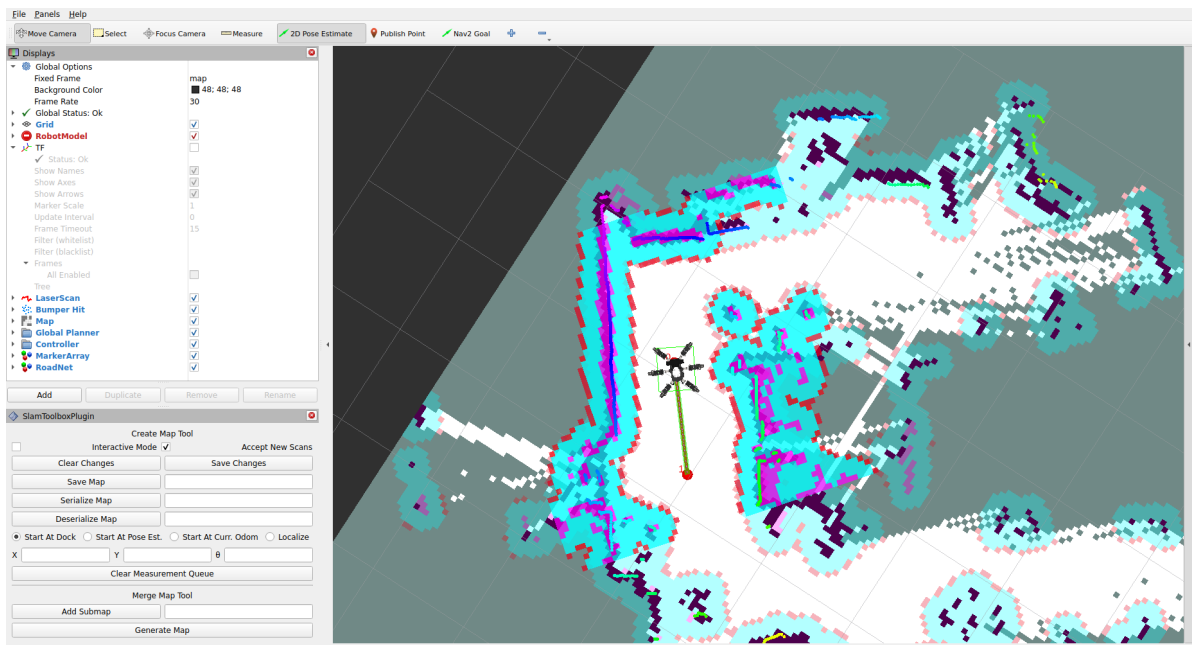


## 2.5 Navigation When the Starting Position is Not on the Road Network

- If the starting position is not on a point or edge of the road network, when executing road network navigation, the robot car will automatically move to the nearest edge of the road network first, and then move along the road network. For example, in the following situation, the car is not in the road network. We observe that the nearest waypoint is 1. Here, we plan to go to waypoint 0, and then publish the start and end point topic.

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [1, 0]}"
```





### 3. Source Code Analysis

- Implementation program source code path:

```
~/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

```
~/code_ws/src/road_net_route/road_net_route/conver_speed.py
```

#### 3.1 Analysis of route\_bridge.py

- In the initialization function of the Route\_Bridge class, a `road_net_nav_id` topic subscriber is created to receive `Int16MultiArray` data (in the format `[start_id, goal_id]`) from the `road_net_nav_id` topic, and directly calls `getAndTrackRoute` to initiate road network navigation:
- Input validation: If the data length is not 2, an error is reported; if it is `[-1, -1]`, navigation is canceled.
- Navigation execution: Call `self.navigators.getAndTrackRoute(start_id, goal_id)` to return a navigation task handle.
- Feedback processing: Loop to get navigation feedback (`feedback`), listen for node switching (`last_node_id / next_node_id`) and rerouting (`rerouted`) events. If rerouted, call `followPath` to update the local path.
- Result processing: Output logs based on `TaskResult` (success/cancel/failure), and publish the result via `/action_feedback`.

```
self.road_net_nav_ID_sub= self.create_subscription(Int16MultiArray,
"road_net_nav_id", self.id_nav_callback, 10)
```

```
def id_nav_callback(self, msg: Int16MultiArray):
    ''' callback function for navigation route messages using IDs '''
    if len(msg.data) != 2:
        self.get_logger().error(Fore.RED+"Invalid ID navigation message
received."+Fore.RESET)
        return
    if msg.data[0]==-1 and msg.data[1]==-1 :
```

```

        self.navigator.cancelTask()
        self.get_logger().info(Fore.YELLOW+"navigation task canceled via
message."+Fore.RESET)
        return
    route_tracking_task = self.navigator.getAndTrackRoute(msg.data[0],
msg.data[1])
    task_canceled = False
    last_feedback = None
    follow_path_task = RunningTask.NONE
    while not self.navigator.isTaskComplete(task=route_tracking_task):
        feedback = self.navigator.getFeedback(task=route_tracking_task)
        while feedback is not None:
            if not last_feedback or (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                self.get_logger().info('Passed node ' +
str(feedback.last_node_id) +
                ' to next node ' + str(feedback.next_node_id) +
                ' along edge ' + str(feedback.current_edge_id) + '.')
            last_feedback = feedback
            if feedback.rerouted: # or follow_path_task == RunningTask.None
                self.get_logger().info('Passing new route to controller!')
                follow_path_task = self.navigator.followPath(feedback.path)
                feedback = self.navigator.getFeedback(task=route_tracking_task)
            if self.navigator.isTaskComplete(task=follow_path_task):
                self.get_logger().info('Controller or waypoint follower server
completed its task!')
                self.navigator.cancelTask()
                task_canceled = True
            while not self.navigator.isTaskComplete(task=follow_path_task) and not
task_canceled:
                time.sleep(0.1)

    result = self.navigator.getResult()
    if result == TaskResult.SUCCEEDED:
        self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)

    self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
    elif result == TaskResult.CANCELED:
        self.get_logger().info(Fore.YELLOW+"navigation task
canceled."+Fore.RESET)
    elif result == TaskResult.FAILED:
        self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)
        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
    else:
        self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)
        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

```



## 3.2 Analysis of conver\_speed.py

- Since the speed topic published by navigation2 in the ros2 kilted version is in `TwistStamped` format, while the MutoRS chassis microROS node accepts the `Twist` format speed topic for chassis control, the two cannot communicate directly. A bridge is needed for speed message format conversion.

We can use the following command to see the difference between the two message types. The main difference is `std_msgs/Header header`.

- `TwistStamped` format speed topic

```
ros2 interface show geometry_msgs/msg/TwistStamped
```

```
# A twist with reference coordinate frame and timestamp
```

```
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
Twist twist
  Vector3 linear
    float64 x
    float64 y
    float64 z
  Vector3 angular
    float64 x
    float64 y
    float64 z
```

- `Twist` format speed topic

```
ros2 interface show geometry_msgs/msg/Twist
```

```
# This expresses velocity in free space broken into its linear and angular parts.
```

```
Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
```

- Here, the initialization function of the `TwistStampedToTwist` class defines a subscriber for the `TwistStamped` format speed topic. Then, in the callback function, the speed in `TwistStamped` format is converted to the `Twist` format speed topic and published.



```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped, Twist
from colorama import Fore
class TwistStampedToTwist(Node):
    def __init__(self):
        super().__init__('conver_speed')
        self.sub = self.create_subscription(
            TwistStamped,
            '/cmd_vel',
            self.callback,
            5
        )
        self.pub = self.create_publisher(
            Twist,
            '/cmd_vel',
            5
        )
        self.get_logger().info(Fore.GREEN + "TwistStamped → Twist The converter
has been started."+Fore.RESET)

    def callback(self, msg:TwistStamped):
        t = Twist()
        t.linear = msg.twist.linear
        t.angular = msg.twist.angular
        self.pub.publish(t)

def main():
    rclpy.init()
    node = TwistStampedToTwist()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

## 4. Navigation Parameter Debugging

Road network navigation parameter file path:

```
~/code_ws/src/road_net_route/config/nav2_kilted.yaml
```

### 4.1 Navigation Controller Debugging

[!TIP]

#### Parameter Adjustment Guide

- If you need to adjust the moving speed and turning angular velocity during navigation, adjust `desired_linear_vel` and `max_angular_accel`.

- If you need to adjust the accuracy of reaching the target point, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`.
- For other parameter adjustments, see the comments below.

```

controller_server:
  ros__parameters:
    # ===== Controller Core Global Parameters
    =====
    # Main loop frequency of the controller (Hz), determines the frequency of
    local path planning and velocity command publishing, needs to match the hardware
    execution frequency
    controller_frequency: 20.0
    # Costmap update timeout (seconds), if the costmap is not updated within this
    time, the controller will stop outputting velocity commands to avoid planning
    based on an outdated map
    costmap_update_timeout: 0.30
    # Minimum velocity threshold in the x direction (forward/backward) (m/s),
    velocity commands below this value will be considered invalid, used to filter out
    minor jitters
    min_x_velocity_threshold: 0.001
    # Minimum velocity threshold in the Y direction (strafe) (m/s), only
    applicable to omnidirectional robots (like Mecanum wheels), can be set to a large
    value (e.g., 0.5) for differential drive robots to disable strafing
    min_y_velocity_threshold: 0.5
    # Minimum velocity threshold for rotation (rad/s), rotation commands below
    this value will be considered invalid, filtering out minor rotational jitters
    min_theta_velocity_threshold: 0.001
    # Controller failure tolerance (meters), when the local planned path
    deviation exceeds this value, the controller execution is judged to have failed,
    triggering replanning
    failure_tolerance: 0.3
    # List of progress checker plugins, used to monitor if the robot is moving
    towards the goal
    progress_checker_plugins: ["progress_checker"]
    # List of goal checker plugins, used to determine if the robot has reached
    the goal pose
    goal_checker_plugins: ["general_goal_checker"]
    # List of controller plugins, the core is the local path tracking algorithm,
    here using the pure pursuit controller corresponding to FollowPath
    controller_plugins: ["FollowPath"]
    # Whether to use real-time priority scheduling for the controller thread
    use_realtime_priority: false
    # Speed limit topic name, the controller will subscribe to the velocity
    commands of this topic to dynamically adjust the maximum linear/angular velocity
    (e.g., slowing down for obstacle avoidance)
    speed_limit_topic: "speed_limit"

    # ===== Progress Checker Parameters (SimpleProgressChecker)
    =====
    progress_checker:
      # Progress checker plugin type, SimpleProgressChecker is a basic
      implementation that monitors robot movement distance and time
      plugin: "nav2_controller::SimpleProgressChecker"
      # Minimum required movement radius (meters), if the robot does not move
      more than this distance within movement_time_allowance, it is judged as no
      progress
      required_movement_radius: 0.05

```

```

# Maximum allowed time without progress (seconds), if this time is
exceeded and required_movement_radius is still not met, the controller will abort
the task (error code 105)
movement_time_allowance: 10.0

# ===== Goal Checker Parameters (SimpleGoalChecker)
=====
general_goal_checker:
# Whether it is a stateful checker. If set to True, it will continuously
monitor the goal pose until the tolerance is met; False means only a single
check
stateful: True
# Goal checker plugin type, SimpleGoalChecker is a basic implementation
that judges whether the position and angle are within the tolerance range
plugin: "nav2_controller::SimpleGoalChecker"
# XY plane position tolerance (meters), when the deviation between the
robot's position and the target position is less than this value, the position is
considered reached
xy_goal_tolerance: 0.08
# Yaw angle tolerance (radians), when the deviation between the robot's
orientation and the target orientation is less than this value, the angle is
considered reached
yaw_goal_tolerance: 0.03

# ===== Pure Pursuit Controller Parameters
(RegulatedPurePursuitController) =====
FollowPath:
# Local path tracking plugin type, RegulatedPurePursuitController is a
pure pursuit algorithm with speed regulation, suitable for differential drive
robots
plugin:
"nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
# Desired linear velocity (m/s), the target linear velocity of the robot
during normal driving, will be dynamically scaled by obstacle avoidance/curvature
regulation
desired_linear_vel: 0.2
# Lookahead distance (meters), the core parameter of the pure pursuit
algorithm, determines the distance of the path point the robot tracks, needs to
match the robot's speed
lookahead_dist: 0.5
# Minimum lookahead distance (meters), prevents the robot from turning
frequently due to a too small lookahead distance
min_lookahead_dist: 0.05
# Maximum lookahead distance (meters), prevents the robot from reacting
sluggishly due to a too large lookahead distance
max_lookahead_dist: 0.9
# Lookahead time (seconds), if use_velocity_scaled_lookahead_dist is
enabled, lookahead_distance = desired_linear_vel * lookahead_time
lookahead_time: 1.5
# Angular velocity for rotating to the target heading (rad/s), the
rotation speed for adjusting the heading after the robot reaches the position
rotate_to_heading_angular_vel: 1.3
# TF transform tolerance (seconds), allows the maximum deviation between
the timestamp of the robot's pose transformation and the current time, to solve
TF delay issues
transform_tolerance: 0.5

```

```
# Whether to use velocity-scaled lookahead distance. If set to True, the
lookahead distance changes dynamically with the robot's speed (the faster the
speed, the farther the lookahead)
use_velocity_scaled_lookahead_dist: false
# Minimum linear velocity when approaching the goal (m/s), when the robot
is close to the goal, it slows down to this value to avoid overshooting the goal
min_approach_linear_velocity: 0.05
# Approach velocity scaling distance (meters), when the robot is closer
than this distance to the goal, it starts to decelerate linearly (from
desired_linear_vel to min_approach_linear_velocity)
approach_velocity_scaling_dist: 1.0
# Whether to enable collision detection. If set to True, the controller
will check for collisions on the lookahead path to prevent the robot from hitting
obstacles
use_collision_detection: false
# Maximum allowed time to collision up to the carrot (seconds), only
effective when use_collision_detection is True, used to judge collision risk
max_allowed_time_to_collision_up_to_carrot: 1.0
# Whether to enable linear velocity scaling based on path curvature. If
set to True, the smaller the turning radius, the lower the linear velocity
use_regulated_linear_velocity_scaling: true
# Whether to enable linear velocity scaling based on the costmap. If set
to True, the closer the obstacle, the lower the linear velocity
use_cost_regulated_linear_velocity_scaling: false
# Minimum turning radius to trigger curvature regulation (meters), when
the turning radius corresponding to the path curvature is less than this value,
deceleration starts
regulated_linear_scaling_min_radius: 0.05
# Minimum speed for curvature regulation (m/s), even if the turning radius
is extremely small, the linear velocity will not be lower than this value
regulated_linear_scaling_min_speed: 0.25
# Whether to use a fixed curvature lookahead distance. If set to True, the
lookahead distance is determined by curvature_lookahead_dist
use_fixed_curvature_lookahead: false
# Curvature lookahead distance (meters), only effective when
use_fixed_curvature_lookahead is True
curvature_lookahead_dist: 1.0
# Whether to enable "rotate to heading then move". If set to True, if the
deviation between the robot's heading and the path direction is too large, it
will rotate first and then move forward
use_rotate_to_heading: true
# Minimum angle to trigger rotation to heading (radians), when the
deviation between the robot's heading and the path direction exceeds this value,
rotation adjustment begins
rotate_to_heading_min_angle: 0.785 # approx. 45 degrees
# Maximum angular acceleration (rad/s²), limits the robot's rotational
acceleration to avoid hardware damage from sharp turns
max_angular_accel: 1.2
# Maximum distance for robot pose search (meters), when the controller
cannot find a matching robot pose on the path, it searches for path points within
this distance
max_robot_pose_search_dist: 10.0
# Whether to interpolate curvature after the goal point. If set to True,
the path after the goal point will have interpolated curvature to prevent the
robot from stopping abruptly after reaching the goal
interpolate_curvature_after_goal: false
```

```

# Cost scaling distance (meters), only effective when
use_cost_regulated_linear_velocity_scaling is True, used to calculate the weight
for obstacle avoidance deceleration
cost_scaling_dist: 0.08

# Cost scaling gain, only effective when
use_cost_regulated_linear_velocity_scaling is True, the larger the gain, the more
obvious the obstacle avoidance deceleration
cost_scaling_gain: 1.0

# Inflation cost scaling factor, used to convert the inflation layer cost
of the costmap into a velocity scaling factor, the larger the value, the more
sensitive the obstacle avoidance in the inflation zone
inflation_cost_scaling_factor: 3.0

```

## 4.2 Obstacle Avoidance and Costmap Debugging

[!TIP]

- If the robot gets too close to obstacles in the map during navigation, you can adjust the inflation area to keep the robot away from obstacles by increasing `inflation_radius` (inflation radius) and `cost_scaling_factor` (the larger the value, the less likely the car is to approach obstacles) in `local_costmap` and `global_costmap`.
- For other parameter adjustments, see the comments below.

```

# Local Costmap Configuration: Short-term costmap for real-time robot obstacle
avoidance, updated by scrolling as the robot moves
local_costmap:
  local_costmap:
    ros__parameters:
      # ===== Costmap Core Global Parameters
      =====

      # Costmap update frequency (Hz), determines the refresh rate of the local
      map, needs to match the sensor frequency
      update_frequency: 5.0

      # Costmap publish frequency (Hz), the frequency of publishing the map to
      visualization tools like RViz, lower than the update frequency can reduce
      bandwidth usage
      publish_frequency: 2.0

      # Global coordinate system of the costmap, the local map uses the odom
      frame
      global_frame: odom

      # Robot base coordinate system, consistent with the robot's center
      coordinate system configured in the hardware
      robot_base_frame: base_link

      # Whether to enable rolling window mode, the core feature of the local
      map: the map scrolls with the robot's movement, only keeping the map of the area
      around the robot
      rolling_window: true

      # Width of the rolling window (meters), the horizontal range of the local
      map
      width: 3

      # Height of the rolling window (meters), the vertical range of the local
      map
      height: 3

```

```

# Resolution of the costmap (meters/pixel), represents the actual distance
represented by each pixel, 0.05 means 5 cm/pixel
resolution: 0.05
# Robot footprint (polygon), uses a list of coordinates to represent the
actual shape of the robot
footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
# Padding distance for the robot footprint (meters), an extra safety
distance extended outside the actual footprint, used as a buffer for obstacle
avoidance
footprint_padding: 0.0616
# List of costmap layer plugins, loaded in order (voxel_layer: voxel
obstacle layer; inflation_layer: inflation layer)
plugins: ["voxel_layer", "inflation_layer"]
# List of costmap filter plugins, used for post-processing the map
(keepout_filter: keepout zone filter)
filters: ["keepout_filter"]
# Whether to always publish the full costmap. If set to True, the full map
is published with every update; False only publishes the changed areas (reducing
bandwidth)
always_send_full_costmap: True
# Service introspection mode, disabled means service introspection is
disabled, optional enabled/verbose
service_introspection_mode: "disabled"

# ===== Filter Plugin: KeepoutFilter =====
keepout_filter:
# Filter plugin type, KeepoutFilter is used to mark keepout zones in the
costmap
plugin: "nav2_costmap_2d::KeepoutFilter"
# Whether to enable keepout zone filtering, KEEPOUT_ZONE_ENABLED is an
external parameter (needs to be assigned true/false at startup)
enabled: KEEPOUT_ZONE_ENABLED
# Subscription topic for keepout zone information, receives the geometry
and position information of keepout zones (e.g., PolygonStamped)
filter_info_topic: "keepout_costmap_filter_info"
# Whether to override the lethal cost of the keepout zone. If set to
True, a custom cost is used instead of the default lethal cost (255)
override_lethal_cost: True
# Custom lethal cost for the keepout zone, 200 means it is lower than
the default lethal cost (255), but still a high-cost keepout zone
lethal_override_cost: 200

# ===== Layer Plugin: InflationLayer =====
inflation_layer:
# Layer plugin type, InflationLayer is used to generate an inflation
area around obstacles to avoid robot collisions
plugin: "nav2_costmap_2d::InflationLayer"
# Cost scaling factor, controls the cost decay rate of the inflation
area (the larger the value, the faster the cost decays, and the smaller the
inflation area)
cost_scaling_factor: 3.0
# Inflation radius (meters), the inflation range around obstacles, the
minimum safe distance from the robot's footprint to the obstacle
inflation_radius: 0.1

# Global Costmap Configuration: Long-term costmap for global robot path planning,
covering the entire environment
global_costmap:

```

```

global_costmap:
  ros__parameters:
    # ===== Costmap Core Global Parameters
    =====

    # Costmap update frequency (Hz), the global map does not need high-
    frequency updates, setting it to 1-2Hz is sufficient
    update_frequency: 1.0
    # Costmap publish frequency (Hz), can be consistent with the update
    frequency
    publish_frequency: 1.0
    # Global coordinate system of the costmap, the global map uses the map
    frame (globally consistent, suitable for path planning)
    global_frame: map
    # Robot base coordinate system, consistent with the local map
    robot_base_frame: base_link
    # Robot radius (replaced by footprint, commented out)
    # robot_radius: 0.22
    # Robot footprint (consistent with the local map)
    footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
    # Padding distance for the robot footprint (consistent with the local map)
    footprint_padding: 0.0616
    # Resolution of the costmap (consistent with the local map to ensure
    coordinate matching)
    resolution: 0.05
    # Whether to track unknown space. If set to True, unexplored areas are
    marked on the map (used for SLAM and unknown environment planning)
    track_unknown_space: true
    # List of costmap layer plugins (static_layer: static layer;
    obstacle_layer: 2D obstacle layer; inflation_layer: inflation layer)
    plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
    # List of costmap filter plugins (keepout_filter: keepout zone filter;
    speed_filter: speed limit filter)
    filters: ["keepout_filter", "speed_filter"]
    # Whether to always publish the full costmap
    always_send_full_costmap: True
    # Service introspection mode
    service_introspection_mode: "disabled"

    inflation_layer:
      # Layer plugin type, InflationLayer is used to generate an inflation
      area around obstacles to avoid robot collisions
      plugin: "nav2_costmap_2d::InflationLayer"
      # Cost scaling factor, controls the cost decay rate of the inflation
      area (the larger the value, the faster the cost decays, and the smaller the
      inflation area)
      cost_scaling_factor: 3.0
      # Inflation radius (meters), the inflation range around obstacles, the
      minimum safe distance from the robot's footprint to the obstacle
      inflation_radius: 0.1

```



